

AFRILANG Stdlib

std/c/grapheuler

Français. Bibliothèque AFRILANG 'std/c/grapheuler' (niveau complexe, catégorie complexe). Elle expose 7 fonction(s) : vertexDegrees, hasEulerCircuit, hasEulerPath, oddDegreeCount, totalDegree, eulerTrailLen, isEulerian.

```
'import "std/c/grapheuler"' · 'use grapheuler'  
- 'vertexDegrees(adj list number, n number) → list number' - 'hasEulerCircuit(adj list number, n number) → bool' - 'hasEulerPath(adj list number, n number) → bool' - 'oddDegreeCount(adj list number, n number) → number' - 'totalDegree(adj list number, n number) → number' - 'eulerTrailLen(adj list number, n number) → number' - 'isEulerian(adj
```

English. AFRILANG library 'std/c/grapheuler' (tier complex, category complex). Exports 7 function(s): vertexDegrees, hasEulerCircuit, hasEulerPath, oddDegreeCount, totalDegree, eulerTrailLen, isEulerian.

```
'import "std/c/grapheuler"' · 'use grapheuler'  
- 'vertexDegrees(adj list number, n number) → list number' - 'hasEulerCircuit(adj list number, n number) → bool' - 'hasEulerPath(adj list number, n number) → bool' - 'oddDegreeCount(adj list number, n number) → number' - 'totalDegree(adj list number, n number) → number' - 'eulerTrailLen(adj list number, n number) → number' - 'isEulerian(adj list number,
```

Catégorie / Category	Modules complexe (c/)	
Niveau / Tier	complexe	June 16, 2026
Fonctions / Functions	7	

Contents

Français

1. Vue d'ensemble

Bibliothèque AFRILANG 'std/c/grapheuler' (niveau complex, catégorie complex). Elle expose 7 fonction(s) : vertexDegrees, hasEulerCircuit, hasEulerPath, oddDegreeCount, totalDegree, eulerTrailLen, isEulerian.

'import "std/c/grapheuler"' · 'use grapheuler'

```
- `vertexDegrees(adj list number, n number) → list number`
- `hasEulerCircuit(adj list number, n number) → bool`
- `hasEulerPath(adj list number, n number) → bool`
- `oddDegreeCount(adj list number, n number) → number`
- `totalDegree(adj list number, n number) → number`
- `eulerTrailLen(adj list number, n number) → number`
- `isEulerian(adj list number, n number) → bool`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/grapheuler"
use grapheuler
```

Niveau : complex · Catégorie : Modules complex (c/)
Domaines avancés – graphes, NLP, réseaux complexes.

3. Référence API

- vertexDegrees(adj list number, n number) → list
- hasEulerCircuit(adj list number, n number) → bool
- hasEulerPath(adj list number, n number) → bool
- oddDegreeCount(adj list number, n number) → number
- totalDegree(adj list number, n number) → number
- eulerTrailLen(adj list number, n number) → number
- isEulerian(adj list number, n number) → bool

4. Exemples d'utilisation

```
import "std/c/grapheuler"
use grapheuler
```

```
create result = vertexDegrees(/* args */)
say result
```

```
create result = hasEulerCircuit(/* args */)
say result
```

```
create result = hasEulerPath(/* args */)
say result
```

```
create result = oddDegreeCount(/* args */)
say result
```

```
create result = totalDegree(/* args */)
say result
```

```
create result = eulerTraillLen(/* args */)
say result
```

5. Bonnes pratiques

- Préférez ``import "std/c/grapheuler"`` en tête de fichier.
- Lancez ``afriLang check`` avant de livrer.
- Combinez avec les modules stdlib de la même catégorie.
- Voir `/stdlib/` pour le source live et les démos playground.
- Signalez les problèmes sur GitHub si une export manque.

6. Annexe — source

module grapheuler

```
function vertexDegrees(adj list number, n number) returns list number
end
```

```
function hasEulerCircuit(adj list number, n number) returns bool
end
```

```
function hasEulerPath(adj list number, n number) returns bool
end
```

```
function oddDegreeCount(adj list number, n number) returns number
end
```

```
function totalDegree(adj list number, n number) returns number
end
```

```
function eulerTraillLen(adj list number, n number) returns number
end
```

```
function isEulerian(adj list number, n number) returns bool
end
end
```

English

1. Overview

AFRILANG library 'std/c/grapheuler' (tier complex, category complex). Exports 7 function(s): vertexDegrees, hasEulerCircuit, hasEulerPath, oddDegreeCount, totalDegree, eulerTrailLen, isEulerian.

```
'import "std/c/grapheuler"' · 'use grapheuler'
```

```
- `vertexDegrees(adj list number, n number) → list number`
- `hasEulerCircuit(adj list number, n number) → bool`
- `hasEulerPath(adj list number, n number) → bool`
- `oddDegreeCount(adj list number, n number) → number`
- `totalDegree(adj list number, n number) → number`
- `eulerTrailLen(adj list number, n number) → number`
- `isEulerian(adj list number, n number) → bool`
```

2. Installation & import

Add the module to your program:

```
import "std/c/grapheuler"
use grapheuler
```

Tier: complex · Category: Complex modules (c/)
Advanced domains – graphs, NLP, complex networks.

3. API reference

- vertexDegrees(adj list number, n number) → list
- hasEulerCircuit(adj list number, n number) → bool
- hasEulerPath(adj list number, n number) → bool
- oddDegreeCount(adj list number, n number) → number
- totalDegree(adj list number, n number) → number
- eulerTrailLen(adj list number, n number) → number
- isEulerian(adj list number, n number) → bool

4. Usage examples

```
import "std/c/grapheuler"
use grapheuler
```

```
create result = vertexDegrees(/* args */)
say result
```

```
create result = hasEulerCircuit(/* args */)
say result
```

```
create result = hasEulerPath(/* args */)
say result
```

```
create result = oddDegreeCount(/* args */)
say result
```

```
create result = totalDegree(/* args */)
say result
```

```
create result = eulerTraillLen(/* args */)
say result
```

5. Best practices

- Prefer ``import "std/c/grapheuler"`` at the top of your file.
- Run ``afriLang check`` before shipping.
- Combine with related stdlib modules from the same category.
- See `/stdlib/` for the live source and playground demos.
- Report issues on GitHub if an export is missing or undocumented.

6. Appendix — source

module grapheuler

```
function vertexDegrees(adj list number, n number) returns list number
end
```

```
function hasEulerCircuit(adj list number, n number) returns bool
end
```

```
function hasEulerPath(adj list number, n number) returns bool
end
```

```
function oddDegreeCount(adj list number, n number) returns number
end
```

```
function totalDegree(adj list number, n number) returns number
end
```

```
function eulerTraillLen(adj list number, n number) returns number
end
```

```
function isEulerian(adj list number, n number) returns bool
end
end
```

Référence rapide / Quick reference

- Import : `import "std/c/grapheuler"`
- Vérification : `afriLang check`
- Documentation : <https://afriLang.dev/stdlib/std/c/grapheuler/>

Source (excerpt)

```
module grapheuler

function vertexDegrees(adj list number, n number) returns list number
end

function hasEulerCircuit(adj list number, n number) returns bool
end

function hasEulerPath(adj list number, n number) returns bool
end

function oddDegreeCount(adj list number, n number) returns number
end

function totalDegree(adj list number, n number) returns number
end

function eulerTraillen(adj list number, n number) returns number
end

function isEulerian(adj list number, n number) returns bool
end
end
```